

Company: BrowserStack

Job Profile: Software Engineer - Backend

Offer Type: Internship (Performance based PPO)

Internship Stipend: 75K per month

CTC: Not Mentioned (But we got to know from seniors that the offer is around **22-22.5 LPA**)

Location: Mumbai (Remote)

Process Date: Jul 23, 2025 (OA)

Jul 31, 2025 (Interview)

JD :  Software Engineer - Backend Intern JD- BrowserStack.pdf

Placement Process:

1. Online Assessment (MCQ Round)
2. Machine Coding Round (Don't be afraid after reading "Machine Coding Round"; see below for what I've written)
3. Interview

1. Online Assessment (MCQ Round) [92 candidates appeared for this round]

Platform : HackerEarth

Eligibility: CE and CSE (For the 2025 passout batch, EXTC was also eligible, so for 2027, anything could happen.)

There was a 7.5+ CGPA criteria to apply.

(Just telling in case you don't know — the college considers CGPA of only 5 semesters till the 6th sem result is out. **In our case, from August 8th, they began considering the 6th semester CGPA as well**)

There were 21 technical MCQs and 50 mins were given, but 15–20 mins were more than enough to complete everything. Since there was a lot of time, I noted down a few points and some less-known theory questions:

1. In VB-style, -1 is true and 0 is false
2. Output-based question on recursion

3. What is Anonymous FTP
4. What is CORS
5. What is lazy loading
6. How to select an HTML tag using ID
7. Questions based on HTTP status codes – so study all status codes
8. DHCP is an application-level protocol
9. One question was on operator precedence – had to evaluate an expression
10. There was a question asking for the order from bottom to top for elements (terms like relative, absolute, z-index were mentioned)
11. Again, a question on order of console.log() means asking for output which involved Promises and setTimeout – basically question on event loop (Study the event loop in JS - one of the favorite interview questions when it comes to JS)
12. Ping does not use port
13. Valid ping command – remember all octets must be numbers between 0 to 255, other options had URLs so were invalid
14. One SQL question on joins – options had actual SQL queries

A key point: They evaluate not just your MCQ score but also shortlist based on your resume, so ensure it highlights solid development skills.

11 candidates were shortlisted for the next round, but out of those, 2 got offers from Barclays and 2 from IDFC. So finally **only 7 candidates** appeared for the next round.

2. **Machine Coding Round** [7 candidates appeared for this round] [Offline] [2hrs]

As I mentioned, don't be afraid after reading "Machine Coding Round." It's just a problem statement, and you need to provide a solution. BrowserStack has been using the same problem statement for the last 3-4 years. The problem involves implementing Socket.IO for real-time updates, so make sure you're comfortable with it.

We were called to BrowserStack's office in Goregaon for the machine coding round and interview.

HR called every selected candidate 2 days before the interview to invite us to their Goregaon office. On the call, she mentioned there would be 2 rounds: machine coding and then an interview. She also said the machine coding problem statement requires knowledge of Socket.IO and suggested practicing for the machine coding round.

Candidates were divided into batches, with 5 candidates per batch and one mentor per batch. On the same day, there were candidates from other colleges too. I think there were about 12 batches in total. I was in Batch 1 with 5 candidates, all from our college. The other 2 candidates from our college were in Batch 2 with 3 candidates from a different college.

Instructions Provided to Us:

What to bring:

- Personal laptop
- Hard copy of resume
- Government-issued photo ID

Ensure the following setup on your laptop before the interview:

- Compiler/interpreter of your preferred programming language
- IDE/code editor of your choice
- Root/admin access to the system
- Latest version of Chrome, Firefox, Safari, or Edge

We were allowed to use our personal laptops, and Google and GPT were permitted.

Problem Statement:

(You'll find it in a GitHub repo too - my solution is linked at the end of this section.)

This problem requires you to implement a log-watching solution (similar to the `tail -f` command in UNIX). However, the log file is hosted on a remote machine (same machine as your server code). The log file is in append-only mode.

You need to implement the following:

- A server-side program to monitor the given log file and stream updates as they happen using Socket.IO for real-time communication. This will run on the same machine as the log file. You can use any programming language for the server.
- A web-based client (accessible via a URL like `http://localhost/log`) that displays updates in the file in real-time, without requiring a page refresh. The page should load once and keep updating. On initial load, the user should see the last 10 lines of the file.

Problem Constraints:

- The server must push updates to clients in real-time using Socket.IO.
- The log file could be several GBs. How do you optimize retrieving the last 10 lines?
- The server should not retransmit the entire file every time; it should only send updates.
- The server must handle multiple clients simultaneously.

- The web page should not stay in a loading state after the first load and should not reload thereafter.
- You may not use off-the-shelf external libraries or tools to read the file or provide tail-like functionalities (except Socket.IO for real-time updates).

You'll be evaluated on code quality, testability, modularity, and handling of corner cases.

In this round, you can ask the mentor any doubts. At the end, the mentor will review your solution and test it thoroughly. For me, the mentor asked me to open the URL in two browsers, perform a hard refresh, and test edge cases. He also used a bash script to simulate automatic updates in the text file, adding `\n`, `\t`, and `\r` characters to check if everything worked correctly.

Tip: Know every single line of your code and why you wrote it, as the mentor will ask about your logic and the reasoning behind specific lines, especially how you implemented Socket.IO for real-time updates.

After this round, we had lunch at their office through their catering service. Then, after lunch, they announced the results, and **only 3 candidates** were selected for the next round.

Sometimes they also ask this PS:

 **Browser automation without using automation frameworks.pdf**

3. Interview Round [3 candidates appeared for this round] [50-55 min in my case]

The interviewer was the Director of Engineering. He kicked things off by introducing himself and then asked about my CET, SSC, and HSC scores, and why I chose SPIT. After that, he asked me to give a quick introduction about myself. Then, he asked about my internship experience, specifically what I did during it and what kind of work I handled. I shared details about my role, the tasks I contributed to, and the impact I made, like leading development or solving specific challenges. Once I was done, he explained the interview flow: first, we'd dive into my projects, and then he'd test my problem-solving skills.

He started with, "Tell me about a project where you faced a lot of difficulties and challenges." So, I began explaining one of my projects, and while I was talking, he jumped in with questions. He asked about specific situations where I hit technical roadblocks, how I resolved them, and how I tackled things overall.

Then came the “why” questions:

- Why did I pick Next.js over React.js for my project? (I used Next.js, so I explained my reasoning—stuff like server-side rendering and better SEO.)
- Why SQL over NoSQL? Then, in SQL, why PostgreSQL over MySQL? What was the reason behind these choices?
- What if I had used a NoSQL database for the same project? How would it have impacted things?

After the “why” questions, he dug deeper into scaling and deployment:

- How would I scale up my existing project?
- What factors did I consider when deploying my project?
- How was I monitoring things post-deployment, and why did I choose Vercel for deployment?
- Since I used NeonDB (PostgreSQL), what features did NeonDB provide that helped throughout my project?

After the project discussion, we moved to a DSA question: [Daily Temperatures](#). It’s a classic Monotonic Stack problem. He asked me to write proper code on paper and do a dry run with the example he gave. I walked through the solution, explaining the stack logic and how it tracks indices to compute the result.

All three of us landed the offer! 🏆 🚀

Most asked questions at BrowserStack : (Questions in Red were asked in my interview)

1. Web and Networking Basics:

- How does the internet work?
- How does an image get downloaded in a browser?
- What happens behind the scenes when you enter a URL in a browser?
- If you type google.com and the page doesn’t load, how would you debug it?
- How does google.com work?
- Describe the journey of a request.
- HTTP vs HTTPS.
- TCP vs UDP.
- Study all HTTP status codes.
- Headers in HTTP requests and responses.

2. Database and System Design:

- HAVING vs WHERE in SQL.
- What is a REST API?
- What is SOAP?
- All types of logs used while building a system.
- Huge file handling: You have a 1TB file and need to count word occurrences with only 8GB of RAM. How would you process it? (MapReduce)

- What are messaging queues?
 - 3. **Concurrency and Processes:**
 - Process vs Thread.
 - Difference between async, concurrency, and parallelism.
 - 4. **Testing and Deployment:**
 - Testing before deployment.
 - How would you design the onboarding flow for your favorite product?
 - 5. **DSA Questions:**
 - [Daily Temperatures \(Monotonic Stack\)](#)
 - [Move Zeroes](#)
 - [Add Two Numbers](#)
 - [Longest Substring Without Repeating Characters](#)
 - [3Sum](#)
-

Useful Tips:

I'm not listing everything you already know—just focusing on some less commonly discussed topics :

- **OOPS:** Study on GeeksforGeeks (GFG) Best Resource for OOPS.
- **Puzzles:** Practice [GFG's Top 100 Puzzles](#).
- **Must-Study Topics:**
 1. **SQL:** Check Drive link PDF for overlooked SQL topics.
 2. **Project Architecture:** Know your project's design and choices.
 3. **Database Scaling & CAP Theorem:** Understand scaling and trade-offs.
 4. **Rate Limiting:** Learn API rate limiting basics.
 5. **Pagination:** Study efficient data handling.
 6. **JWT Authentication:** Know how JWT works for APIs.
 7. **JavaScript:** Master Event Loop, Fetch API, debouncing, throttling.
 8. **Node.js Concurrency:** Explain how single-threaded Node.js handles concurrency.
 9. **MVC Architecture:** Understand Model-View-Controller basics.
 10. **React.js:** Study Virtual DOM, state management, hooks, component lifecycle.
 11. **Java:** Learn JDK, JRE, JVM flow, garbage collection, stack/heap memory.
- Check out the [Drive Link](#) for additional materials like PDFs.

Best of Luck! 🤘

Feel free to reach out:

- WhatsApp: 8928542346 (I prefer WhatsApp)
- LinkedIn: <https://www.linkedin.com/in/kunal-bhoi-7657632b0/>